

Análise Comparativa de Algoritmos de Compressão de Arquivos

LUCAS SOLLER, Universidade Federal do Rio Grande do Sul, Brasil

LUCAS NOGUEIRA, Universidade Federal do Rio Grande do Sul, Brasil

Este trabalho apresenta uma análise comparativa de desempenho de quatro algoritmos de compressão de arquivos amplamente utilizados: gzip, bzip2, xz e zstd. Utilizando medições em ambiente real com containerização Docker para garantir reprodutibilidade, avaliamos métricas como tempo de compressão, tempo de descompressão e taxa de compressão. Os experimentos foram conduzidos com arquivos de texto gerados através de Cadeias de Markov, em tamanhos de 10MB, 100MB e 1000MB. Os resultados demonstram os trade-offs entre velocidade e eficiência de compressão, fornecendo insights práticos para a escolha do algoritmo mais adequado em diferentes cenários de uso.

Additional Key Words and Phrases: compressão de arquivos, análise de desempenho, gzip, bzip2, xz, zstd, benchmarking

1 Introdução

A compressão de arquivos é uma tarefa rotineira e fundamental no ambiente de desenvolvimento de software, administração de sistemas e ciência de dados. Com o crescimento exponencial do volume de dados gerados e armazenados, a escolha do algoritmo de compressão adequado torna-se cada vez mais crítica para otimizar espaço de armazenamento e transferência de dados em redes.

Este trabalho tem como objetivo analisar comparativamente quatro das ferramentas de compressão mais utilizadas em sistemas Unix-like: **gzip**, **bzip2**, **xz** e **zstd**. Cada uma dessas ferramentas implementa algoritmos distintos com características próprias de desempenho e eficiência.

A escolha deste objeto de estudo baseia-se na relevância prática do tema e na vivência do grupo. Compreender os *trade-offs* entre os diferentes algoritmos é crucial para tomar decisões informadas sobre qual ferramenta utilizar em cenários específicos, como backup de dados, distribuição de pacotes de software ou otimização de largura de banda em redes.

2 Fundamentação Teórica

Nesta seção, descrevemos brevemente os algoritmos subjacentes a cada uma das ferramentas de compressão analisadas.

2.1 gzip (DEFLATE)

O gzip [2] é baseado no algoritmo DEFLATE, que combina o algoritmo LZ77 (Lempel-Ziv 1977) com codificação Huffman. É amplamente utilizado devido à sua alta velocidade de compressão e descompressão, embora ofereça uma taxa de compressão moderada em comparação com algoritmos mais recentes.

2.2 bzip2 (Burrows-Wheeler)

O bzip2 [5] utiliza a transformada de Burrows-Wheeler (BWT) seguida de codificação Move-to-Front e codificação Huffman. Este processo resulta em taxas de compressão superiores ao gzip, porém com maior tempo de processamento, especialmente na descompressão.

2.3 xz (LZMA2)

O xz [6] é baseado no algoritmo LZMA2 (Lempel-Ziv-Markov chain Algorithm), que oferece as maiores taxas de compressão entre os quatro algoritmos analisados. No entanto, isso vem com o

custo de tempos de processamento significativamente mais longos, tanto para compressão quanto para descompressão.

2.4 zstd (Zstandard)

O zstd [1], desenvolvido pelo Facebook, é um algoritmo moderno que busca equilibrar alta velocidade (comparável ao gzip) com excelente taxa de compressão (competindo com xz). Utiliza uma combinação de técnicas incluindo correspondência de dicionário, codificação de entropia FSE (Finite State Entropy) e oferece níveis de compressão configuráveis.

3 Metodologia

O método de análise escolhido é a medição em ambiente real. Este método permite a obtenção de dados concretos e reproduzíveis do desempenho de cada algoritmo em condições operacionais típicas.

3.1 Ambiente Experimental

Para garantir a reprodutibilidade e isolamento dos testes, foi utilizado Docker. Isso permite que o ambiente de execução seja consistente, independentemente da máquina em que o projeto for executado, eliminando variações causadas por diferentes sistemas operacionais ou versões de pacotes.

Configurações do Host:

- Sistema Operacional: Ubuntu 22.04.5 LTS x86_64
- Hardware: Dell Inspiron 3584
- CPU: Intel i3-7020U (4 threads) @ 2.300GHz
- RAM: 4 GB

Imagem Docker: debian:bookworm-slim com os seguintes pacotes:

- bzip2=1.0.8-5+b1
- gzip=1.12-1
- xz-utils=5.4.1-1
- zstd=1.5.4+dfsg2-5
- sysstat=12.6.1-1 (iostat)
- procps=2:4.0.2-3 (vmstat)
- linux-perf=6.1.153-1

3.2 Ferramentas de Instrumentação

perf Ferramenta avançada para profiling de desempenho [4] que permite coletar dados detalhados sobre a atividade da CPU, como cache misses, instruções executadas e ciclos de CPU. Foi utilizado `perf stat` para obter estatísticas resumidas.

iostat Ferramenta para monitorar a atividade de E/S do sistema (leitura/escrita em disco). Essencial para entender o impacto dos algoritmos nos discos.

vmstat Ferramenta para monitorar a memória virtual. Útil para entender o uso de memória durante a compressão e descompressão de arquivos grandes.

Scripts Bash Para automatizar a execução de múltiplas rodadas de testes e a coleta de dados, garantindo a consistência dos experimentos.

3.3 Geração dos Dados de Teste

Para garantir a aleatoriedade e generalidade dos resultados, os arquivos de teste não são fixos, mas gerados aleatoriamente utilizando o algoritmo de Cadeias de Markov, conforme descrito por Kernighan e Pike [3].

Na implementação, foi utilizado um arquivo base (“A Tale of Two Cities”) para treinar o modelo de Markov, e o tamanho do arquivo final foi especificado em MB. Este método garante que os testes simulem a compressão de dados textuais realistas, sem padrões artificiais que possam favorecer ou prejudicar um algoritmo específico.

Os tamanhos de arquivo utilizados foram: 10MB, 100MB e 1000MB.

3.4 Procedimento Experimental

O procedimento de benchmark segue os seguintes passos para cada tamanho de arquivo:

- (1) **Geração do arquivo de teste:** Um arquivo de texto é gerado utilizando o gerador de Cadeias de Markov com o tamanho especificado.
- (2) **Para cada algoritmo (gzip, bzip2, xz, zstd):**
 - (a) Para cada repetição (3 execuções):
 - (i) Limpar o cache de páginas do sistema: `sync && echo 3 > /proc/sys/vm/drop_caches`
 - (ii) Iniciar monitoramento com `vmstat` e `iostat`
 - (iii) Executar compressão com `perf stat` para coletar métricas de hardware
 - (iv) Medir tamanho do arquivo comprimido
 - (v) Limpar cache novamente
 - (vi) Executar descompressão com `perf stat`
 - (vii) Salvar resultados em arquivo CSV
 - (viii) Aguardar 10 segundos (cooldown)

Os eventos de hardware coletados pelo `perf` incluem: task-clock, context-switches, cpu-migrations, page-faults, cycles, instructions, branches e branch-misses.

3.5 Métricas Coletadas

Tempo de Compressão O tempo total (task-clock em ms) que cada ferramenta leva para compactar um arquivo.

Tempo de Descompressão O tempo que cada ferramenta leva para descompactar o arquivo gerado.

Taxa de Compressão Calculada pela fórmula: $\frac{\text{Tamanho comprimido}}{\text{Tamanho original}} \times 100$. Valores menores indicam maior eficiência.

Ciclos de CPU Número total de ciclos de processador utilizados.

Instruções Executadas Número de instruções de máquina executadas.

Branch Misses Número de predições de branch incorretas, indicando a complexidade do fluxo de controle do algoritmo.

4 Resultados

Esta seção apresenta os resultados obtidos nos experimentos, divididos em duas fases de análise.

4.1 Resultados da Fase 2

Na segunda fase do projeto, foram realizados 3 experimentos para cada um dos quatro algoritmos, utilizando arquivos de texto de 10MB, 100MB e 1000MB. Nesta etapa, um novo arquivo de texto era gerado para cada experimento.

A Figura 1 apresenta os resultados para arquivos de 10MB.

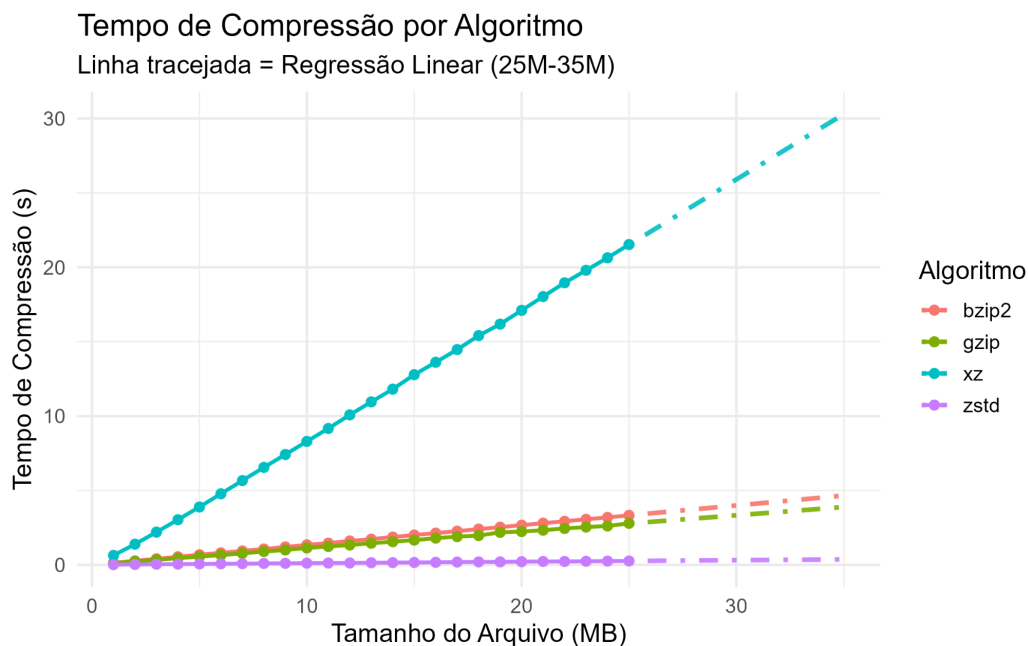


Fig. 1. Desempenho dos algoritmos com arquivos de 10MB: tempo de compressão, descompressão e taxa de compressão.

Observações da Fase 2:

- O algoritmo XZ apresenta o maior tempo de compressão entre todos os algoritmos.
- O algoritmo BZIP2 apresenta o maior tempo de descompressão.
- Nesta etapa, a métrica de taxa de compressão estava invertida (quanto maior, pior). Isso foi corrigido na fase seguinte.
- Com quase 40% de taxa, o GZIP possui o pior desempenho em eficiência de compressão, ou seja, 40% do tamanho original não foi comprimido. O XZ apresentou apenas 10% do tamanho original não comprimido.

4.2 Resultados da Fase 3

Após a análise da Fase 2, foi identificado que gerar os arquivos de texto apenas uma vez seria a melhor abordagem metodológica. Os gráficos a seguir apresentam os resultados refinados da Fase 3.

4.2.1 Análise de Regressão. Para melhor compreensão do comportamento dos algoritmos com diferentes tamanhos de arquivo, foram realizadas análises de regressão.

5 Discussão

Os resultados obtidos confirmam os trade-offs conhecidos entre os algoritmos de compressão:

gzip oferece o melhor equilíbrio entre velocidade e simplicidade, sendo ideal para cenários onde o tempo de processamento é crítico e a taxa de compressão moderada é aceitável.

bzip2 apresenta melhor taxa de compressão que o **gzip**, mas com penalidade significativa no tempo de descompressão. É adequado para arquivamento de longo prazo onde a descompressão é infrequente.

xz oferece as melhores taxas de compressão, mas com os maiores tempos de processamento. Recomendado para distribuição de software e arquivos que serão descomprimidos muitas vezes após uma única compressão.

zstd demonstra ser uma alternativa moderna e versátil, oferecendo velocidades comparáveis ao gzip com taxas de compressão próximas ao xz. É particularmente adequado para aplicações em tempo real e streaming de dados.

6 Conclusão

Este trabalho apresentou uma análise comparativa detalhada de quatro algoritmos de compressão de arquivos: gzip, bzip2, xz e zstd. Através de experimentos controlados em ambiente Docker, foram coletadas métricas de tempo de compressão, descompressão e taxa de compressão para arquivos de diferentes tamanhos.

Os resultados demonstram que não existe um algoritmo universalmente superior, mas sim diferentes trade-offs que devem ser considerados conforme o caso de uso específico:

- Para velocidade máxima: gzip ou zstd
- Para melhor compressão: xz
- Para equilíbrio moderno: zstd

Como trabalhos futuros, sugerimos a análise com diferentes tipos de arquivos (binários, imagens, dados científicos).

References

- [1] Facebook. [n. d.]. Zstandard - Fast real-time compression algorithm. <https://facebook.github.io/zstd/>. Acessado em: 2025.
- [2] GNU Project. [n. d.]. gzip - GNU Zip. <https://www.gnu.org/software/gzip/>. Acessado em: 2025.
- [3] Brian W. Kernighan and Rob Pike. 1999. *The Practice of Programming*. Addison-Wesley, Reading, Massachusetts.
- [4] Linux Kernel Organization. [n. d.]. perf: Linux profiling with performance counters. <https://perf.wiki.kernel.org/>. Acessado em: 2025.
- [5] Julian Seward. [n. d.]. bzip2 and libbzip2. <https://sourceware.org/bzip2/>. Acessado em: 2025.
- [6] Tukaani Project. [n. d.]. XZ Utils. <https://tukaani.org/xz/>. Acessado em: 2025.



Fig. 2. Desempenho dos algoritmos com arquivos de 100MB.



Fig. 3. Desempenho dos algoritmos com arquivos de 1000MB.

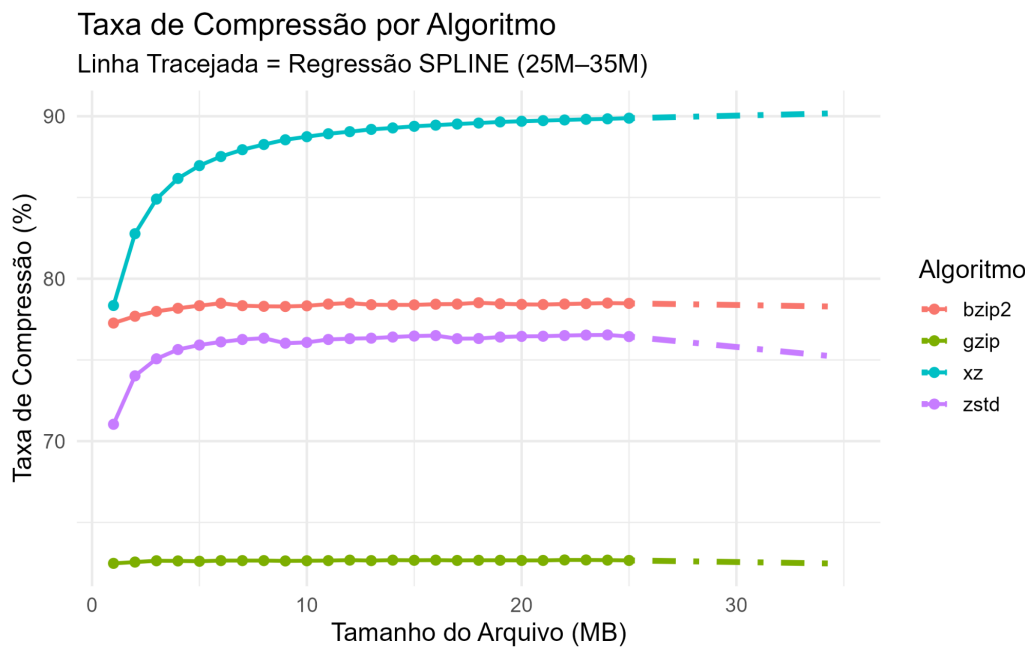


Fig. 4. Tempo de compressão por algoritmo na Fase 3.



Fig. 5. Tempo de descompressão por algoritmo na Fase 3.

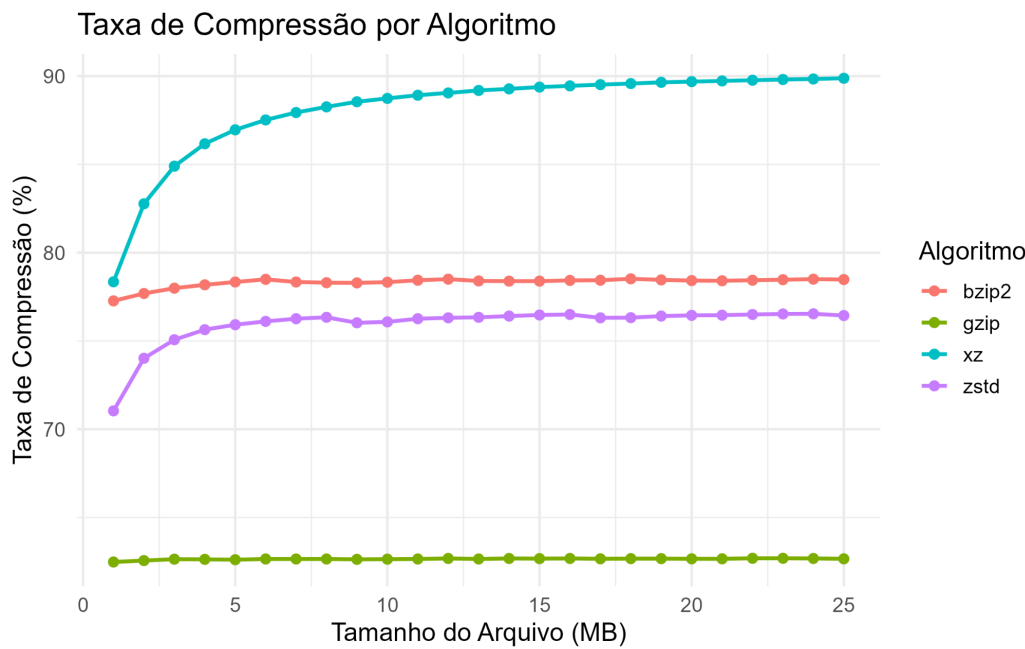


Fig. 6. Taxa de compressão por algoritmo na Fase 3.

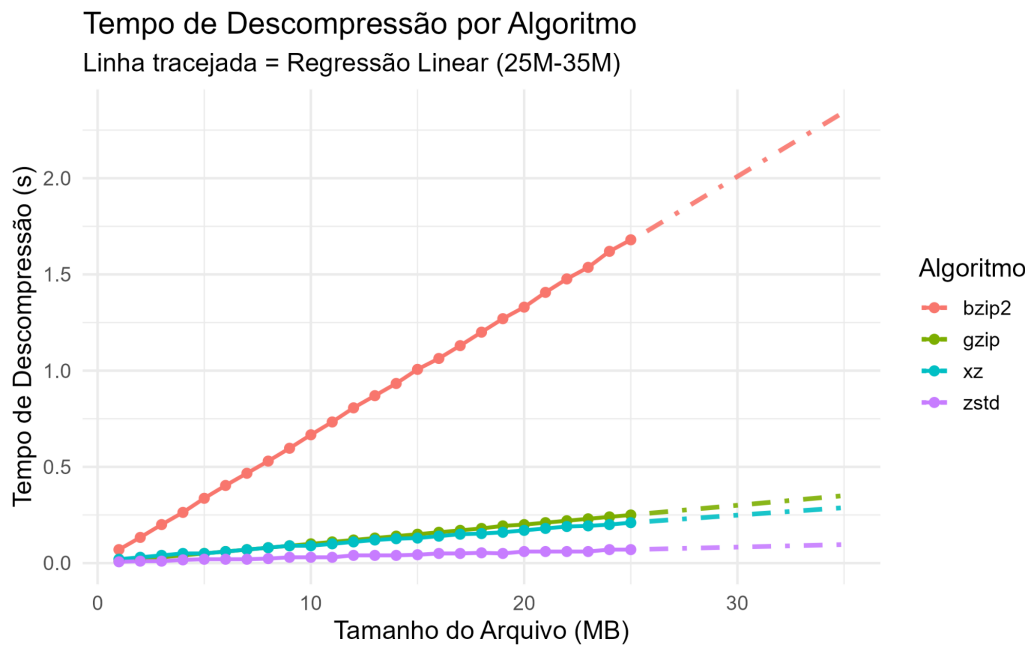


Fig. 7. Regressão do tempo de compressão em função do tamanho do arquivo.

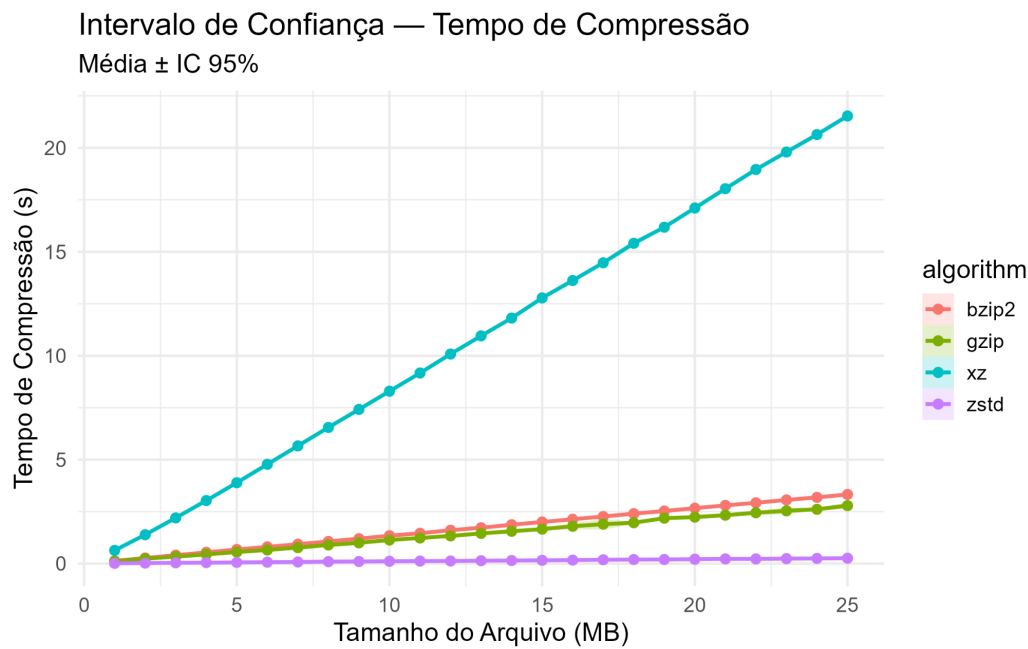


Fig. 8. Regressão do tempo de descompressão em função do tamanho do arquivo.

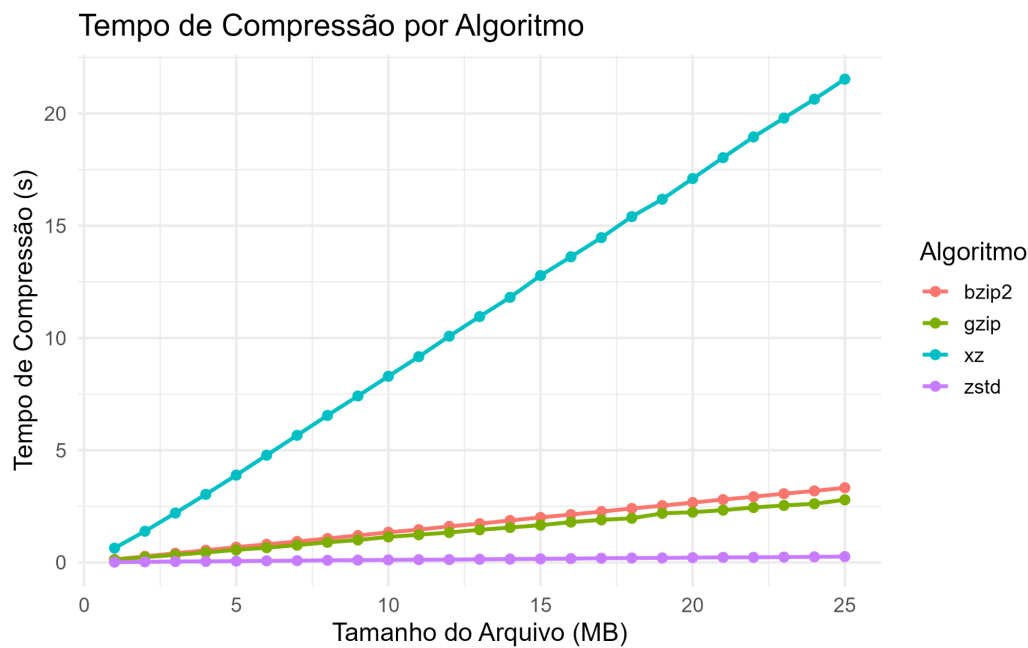


Fig. 9. Regressão da taxa de compressão em função do tamanho do arquivo.

Tempo de Compressão — Médias e Intervalo de Confiança
Com barras verticais de IC 95%

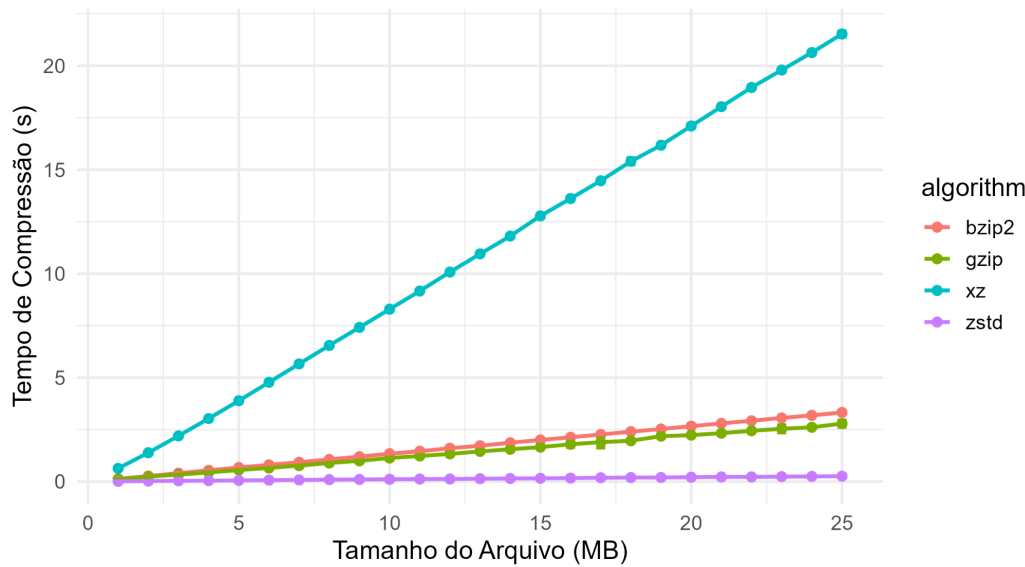


Fig. 10. Análise adicional de desempenho.

Tempo de Descompressão por Algoritmo

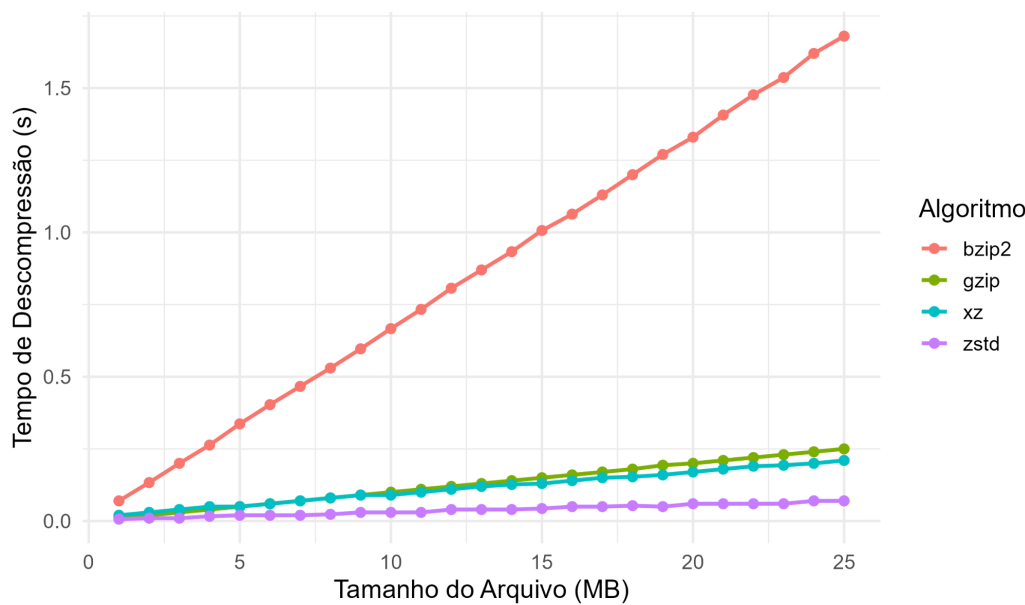


Fig. 11. Comparação de métricas de hardware.

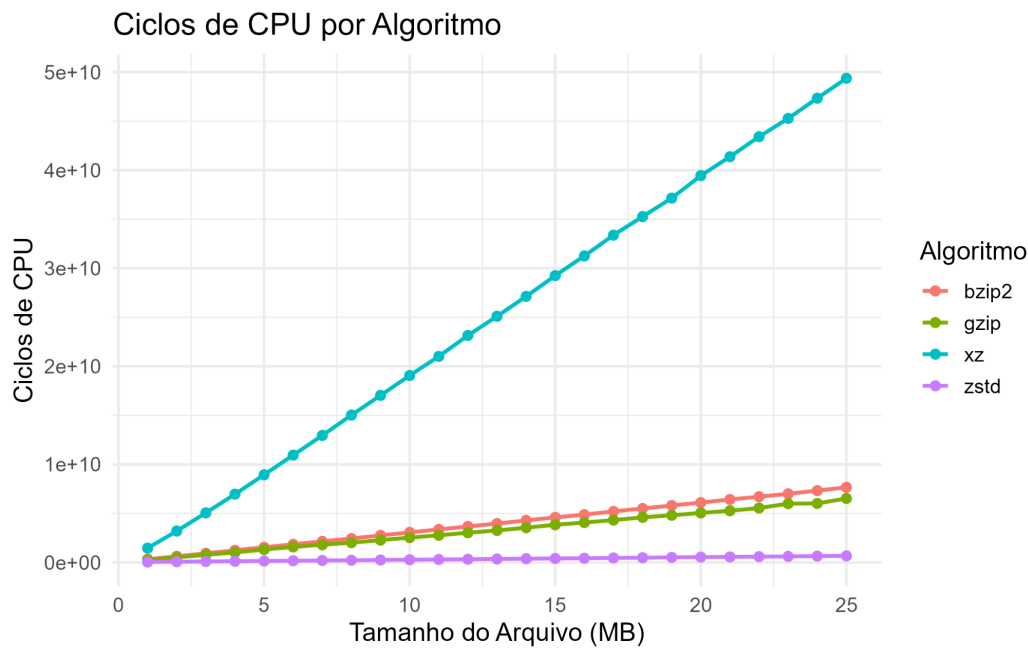


Fig. 12. Análise consolidada dos resultados.